Prompt: página “Robô SDR” no Command Center

Para colar no chat que cuida do Command Center · 12/10/2026 · repositório UraceUs/Uraceagent

Como usar: copie tudo a partir de “Repositório UraceUs/Uraceagent…” até o fim e cole no chat do Command Center. Ele abre PR; depois do merge, o botão “Atualizar sistema” do painel põe no ar.

```
Repositório UraceUs/Uraceagent, branch de trabalho claude/configurar-open-claw-ooqo8x (é a que o VPS roda). Crie uma branch a partir dela, abra PR para ela, siga o template .github/pull_request_template.md e o CI (ruff + pytest, tsc + oxlint + build, e2e). Não mexa em nada da ponte além do que está descrito; não toque em chaves nem no bridge.env fora da linha já tratada por command_center/api/sdr.py.

OBJETIVO
Uma página nova no Command Center, "Robô SDR" (rota /sdr), onde o comercial gere o robô: ver o estado, ligar/desligar, escrever instruções de como o robô deve responder, relatar problemas e acompanhar o que o robô decidiu. Pedido do dono em 12/10/2026: "uma sessão robô SDR onde ele consiga ligar e desligar, passar o que precisa ser feito pelo robô, reportar alguma coisa, gerir o robô". O comercial é quem manda nessas instruções: elas pesam mais que a persona padrão, com duas exceções fixas abaixo (preço e travas).

O QUE JÁ EXISTE (reaproveite, não duplique)
- command_center/api/sdr.py: GET/PUT /ops/api/sdr com o nível do SDR (observar | organizar | atender) gravado em ~/.urace/bridge.env e reinício da ponte via ~/.urace/sdr.request + unit urace-sdr.path (já instalada no VPS). Só ADMIN. Mantenha esse endpoint e esse mecanismo.
- command_center/web/src/pages/CRM.tsx, função InterruptorChase: hoje um checkbox (ligado = atender, desligado = organizar). Mova esse bloco para a página nova e troque por três opções, com explicação de cada uma:
  observar = decide e registra, não escreve no Kommo e não responde;
  organizar = etiqueta, nota, tarefa e etapa no Kommo, não responde ao lead;
  atender = organiza e responde (o robô falando com o lead).
  O estado real hoje no VPS é "observar" (GET /ops/api/sdr devolve nivel=observar, ponte_no_ar=true, instalado=true). Não mude o nível no deploy.
- A ponte de vendas (salesagent/bridge) grava a auditoria em ~/.urace/salesbridge.db, tabela audit (ts, lead_id, kind, detail). As decisões do SDR têm kind='sdr' e detail em JSON (acao, motivo, moveu, tags, nota, tarefa, segurado, sem_regra_2, nao_marcou, ignorado). Já está mascarada (sem token nem telefone).
- Regras do SDR e decisões do dono: salesagent/docs/sdr.md e brain/08_DECISOES/D-2026-09-25 - Chase reorganizado como SDR.md. Decisão de 06/10 (brain/08_DECISOES/D-2026-10-06 - Respostas do dono no painel de decisoes.md): atendimento humano segunda a sexta, 8h às 18h (Orlando).
- Convenções do painel: FastAPI em command_center/api/*.py com APIRouter(prefix="/ops/api/..."), auth.exige("ADMIN"|"MANAGER"|"OPERATOR"), helpers command_center/db (um, todos, inserir, atualizar, auditar, get_db), schema em command_center/db/schema.sql (CREATE TABLE IF NOT EXISTS, comentários com a data e a frase do dono), registrar o router em command_center/api/main.py. Front em React: página em command_center/web/src/pages/, rota em App.tsx dentro do Shell, item no menu em components/Shell.tsx, componentes de components/ui (PageHeader, Section, Chip, Banner, Empty, Loading, ErrorState), useGet/api de ../api, toast de components/Toast, todo texto visível em tr("...") com a tradução em i18n/en.json. Datas sempre em America/New_York. Testes em command_center/tests (pytest com conftest.py; veja test_cofre.py e test_crm.py como modelo).

A PÁGINA /sdr, de cima para baixo
1. Cabeçalho "Robô SDR" com chips: nível atual; ponte no ar/fora do ar; "aplicando…" quando reiniciando; decisões nas últimas 24 h (contagem da auditoria kind='sdr'); última decisão (hora). Mais uma linha: "Horário de atendimento humano: seg–sex 8h–18h (Orlando)".
2. Card "Ligar e desligar": as três opções (radio), só ADMIN muda; os outros papéis veem. Ao trocar, confirmação em uma frase do que vai acontecer; depois do PUT, toast e recarga a cada 15 s até "reiniciando" voltar a false.
3. Card "Instruções para o robô" (MANAGER e ADMIN escrevem; todos leem):
  - lista das instruções ativas, mais novas primeiro, cada uma com categoria, texto, quem escreveu, quando, e os botões "Arquivar" e "Marcar como aplicada no Kommo".
  - formulário: categoria (tom | idioma | preço e condições | horário | passar para pessoa | follow-up | canal | nunca dizer | outro) + texto livre (até 1.000 caracteres).
  - aviso fixo acima do formulário: "Preço vem da tabela do sistema (ratecard). Uma instrução não muda preço; se o preço mudou, avise o dono." Se o texto contiver "$" ou "preço"/"price", mostrar esse aviso em destaque, mas aceitar a instrução.
  - botão "Gerar texto para o agente do Kommo": junta as instruções ativas num bloco de texto pronto para colar na tela do agente de IA do Kommo (o agente "Chase" lá só é editado pela tela do Kommo; cada diretriz lá cabe em 300 caracteres, então quebre em linhas de até 300). Mostra num textarea
```

com "Copiar".
4. Card "Relatos" (qualquer papel escreve): "o robô respondeu errado", "preço errado", "ignorou o lead", "outro", com texto livre e, se quiser, o id ou nome do lead. Lista com status aberto/resolvido e botão "Resolver" (MANAGER+). Um relato novo acende o contador no menu.
5. Card "O que o robô decidiu": as últimas 50 linhas da auditoria kind='sdr', em tabela (hora em Orlando, lead, ação, motivo, o que fez ou faria, nível). Filtro por lead e por ação. Se o banco da ponte não existir no servidor, mostrar "ponte sem auditoria" em vez de erro.

BACKEND
- Tabelas novas em schema.sql:
  sdr_instrucoes(id, categoria, texto, status 'ativa'|'aplicada'|'arquivada', created_by, created_at, applied_at, applied_by, archived_at, archived_by)
  sdr_relatos(id, tipo, texto, lead_ref, status 'aberto'|'resolvido', created_by, created_at, resolved_at, resolved_by)
- Endpoints em command_center/api/sdr.py (mesmo router):
  GET /ops/api/sdr/painel → estado (o mesmo de GET /ops/api/sdr) + contagens + horário humano.
  GET/POST /ops/api/sdr/instrucoes; PATCH /ops/api/sdr/instrucoes/{id} (status). POST e PATCH: MANAGER+. Cada escrita chama auditar(..., "sdr.instrucao", ...).
  GET /ops/api/sdr/instrucoes/texto-kommo → texto pronto para o agente do Kommo (string).
  GET/POST /ops/api/sdr/relatos; PATCH /ops/api/sdr/relatos/{id}. POST: qualquer usuário logado. Auditar "sdr.relato".
  GET /ops/api/sdr/auditoria?lead=&acao=&n=50 → lê ~/.urace/salesbridge.db em modo só leitura (sqlite3 com mode=ro, como conectar_somente_leitura faz), kind='sdr', ordena por ts desc. Sem o arquivo: {"itens": [], "motivo": "ponte sem auditoria"}.
- Toda vez que uma instrução é criada, aplicada ou arquivada, gravar o arquivo ~/.urace/sdr_instrucoes.md com as instruções ativas (uma por linha, "- [categoria] texto"), com escrita atômica (tmp + os.replace, como gravar_nivel faz). É por esse arquivo que a ponte lê as instruções; não mexa no bridge.env para isso.
- Contador no menu: incluir "sdr_relatos_abertos" no payload do dashboard que o Shell já usa (o mesmo lugar de crm_pending) e mostrar no item "Robô SDR" (grupo "Vendas", logo abaixo de "Chat do Kommo", ícone "spark").

PONTE (mudança pequena e única, em salesagent/bridge/app.py, função run_agent)
- Se existir URACE_DIR/sdr_instrucoes.md e tiver conteúdo, injetar no bloco [SYSTEM] do turno, antes de "Memória do lead:", o trecho:
  "Instruções do comercial (valem acima da persona, exceto preço, que vem da tabela, e as travas de segurança):\n<conteúdo>\n"
- Ler o arquivo a cada turno (é pequeno); se falhar a leitura, seguir sem ele e registrar state.log("error", lead_id, "sdr_instrucoes: ...").
- Isso só tem efeito no nível atender (nos outros a ponte não responde). Cobrir com um teste em salesagent/tests (o padrão são scripts com check(); veja test_sdr.py) que prova que o texto entra no prompt quando o arquivo existe e não entra quando não existe.

TESTES
- command_center/tests/test_sdr_painel.py: CRUD de instruções e relatos com os papéis certos (OPERATOR não cria instrução; qualquer um cria relato), arquivo sdr_instrucoes.md escrito e reescrito, texto-kommo quebrando em linhas de até 300 caracteres, auditoria lendo um salesbridge.db de teste e devolvendo "ponte sem auditoria" quando não existe, dashboard com sdr_relatos_abertos.
- e2e: só se a página mexer em tela compartilhada; aqui basta o teste de API.
- Rodar ruff check ., python -m pytest -q, npm run build e npx oxlint src antes do PR.

DEPLOY
- Nada de variável nova. O arquivo ~/.urace/sdr_instrucoes.md nasce na primeira instrução. A ponte precisa reiniciar uma vez depois do merge para carregar a mudança em run_agent (o botão "Atualizar sistema" já faz isso). Descreva isso na seção Deploy do PR.

FORA DESTE PR (só anote no PR)
- O agente de IA "Chase" dentro do Kommo não tem API: as instruções chegam lá pelo botão "Gerar texto para o agente do Kommo" e por quem cola na tela do Kommo. Hoje ele está desligado por decisão do dono.
- Subir o nível para organizar ou atender continua decisão do dono, pela própria página.